

Fitness Battle App

VOZ

Build Your Body. Earn Your Streak.

Mobile Application Documentation

Flutter Development Track — Graduation Project Documentation

Table of Contents

- 1. Project Proposal & Overview 3
- 2. Project Plan..... 3
- 3. Requirements Gathering 4
- 4. System Analysis & Design (Flutter Architecture) 5
- 5. UI/UX Design & Prototyping 7
- 6. Implementation..... 8
- 7. Testing & Quality Assurance..... 8
- 8. Deployment & Execution 9
- 9. Final Presentation & Reports 9

1. Project Proposal & Overview

1.1 Overview

VOZ is a gamified fitness and wellness mobile application built with Flutter. The app guides users through structured workout programs, tracks localized nutrition choices, and rewards consistency through an XP, points, and leveling system. VOZ turns a personal fitness journey into a game-like progression: users unlock levels, protect daily streaks, and customize their profile with earned avatars.

1.2 Objectives

- Provide a structured, level-based workout system suited to Beginner, Intermediate, and Advanced users.
- Encourage daily consistency through streaks, XP rewards, and a visual level-progression timeline.
- Offer localized, budget-aware nutrition guidance priced in Egyptian Pounds (EGP).
- Increase user retention through gamification mechanics — points, avatar unlocks, and milestone celebrations.
- Deliver a smooth, native-feeling cross-platform experience using a single Flutter codebase.

1.3 Scope

The current scope covers the full mobile client built in Flutter: onboarding and authentication, workout and timer logic, a smart nutrition library, profile and avatar-store management, and a gamified dashboard with level progression. Backend/server concerns are limited to what the client consumes (authentication, meal data, and progress persistence) and are treated as an external API contract rather than a separate deliverable.

2. Project Plan

2.1 Milestones & Timeline

Phase	Milestone	Key Output
Phase 1	Planning & Requirements	Feature list, user stories, folder architecture agreed
Phase 2	UI/UX & Theming	Wireframes, AppColors theme, shared widget library
Phase 3	Feature Implementation	Onboarding, Workout, Nutrition, Profile, Dashboard modules
Phase 4	Integration & State Management	Cross-feature navigation, shared state, XP/points sync
Phase 5	Testing & Polish	Manual QA, bug fixes, UI refinement
Phase 6	Final Build & Submission	Signed APK, documentation, demo video

2.2 Deliverables

- Structured, feature-based Flutter source code hosted on GitHub.
- Shared design system (colors, typography, reusable widgets).
- Working APK build demonstrating the complete user flow.
- Technical documentation, user manual, and final presentation.

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Interest / Need
End users	A motivating, easy-to-follow fitness and nutrition companion
Development team	A clean, feature-based Flutter architecture that scales
Course supervisor	Evidence of proper planning, design, and Flutter implementation

3.2 User Stories & Use Cases

As a...	I want to...	So that...
New user	pick my fitness level during onboarding	my workouts match my current ability
Returning user	see today's workout at a glance	I know exactly what to train without extra taps
Active user	start a timer for each exercise	I stay on pace and earn XP on completion
Budget-conscious user	see meal prices in EGP with calories	I can plan meals that fit my budget and goals
Motivated user	track my streak and level progress	I stay consistent and see visible progress
Customizing user	spend earned points on avatars	my profile reflects my achievements

3.3 Functional Requirements

Onboarding & Authentication

- Multi-page onboarding using a PageView, ending with a fitness-level selection step.
- Sign In, Create Account, and Reset Password screens with full form validation.

Workout System & Timer

- Daily workout list grouped by targeted muscle group, with a Rest Day state.

- Exercise detail view with a video tutorial placeholder and a start/stop timer.
- XP and points awarded automatically when a workout is completed.

Smart Nutrition & Meal Library

- Browseable meal list with price (EGP) and calorie count per item.
- Meal detail screen with macro breakdown, preparation steps, and an "eaten" toggle.

Profile, Avatar Store & Settings

- Profile stats: active days, total points, workouts completed, current level.
- Avatar store with a points-based purchase and unlock flow.
- Settings for notifications, theme, app version, and logout.

Dashboard & Level Progression

- Home dashboard with a circular progress ring showing level completion percentage.
- Streak counter and a weekly training checklist.
- A vertical "My Journey" timeline with time-gated level unlock logic.

3.4 Non-Functional Requirements

Category	Requirement
Performance	Screen transitions and timer updates render smoothly at 60fps
Usability	Core actions reachable within two taps from the dashboard
Consistency	All screens use the shared AppColors theme and typography
Reliability	Streaks, XP, and points persist accurately across sessions
Scalability	Feature-based folder structure allows new modules without refactors

4. System Analysis & Design (Flutter Architecture)

4.1 Software Architecture Overview

VOZ follows a feature-based (modular) architecture rather than a single monolithic structure. Each feature owns its screens, widgets, and local state, while shared resources — theme, common widgets, and core services — live in a central core layer. This keeps modules independent enough for parallel development while sharing one visual language.

4.2 Folder Structure

Path	Responsibility
lib/core/theme/app_colors.dart	Global color palette and theming used across every feature
lib/core/widgets/	Shared, reusable UI components (cards, buttons, progress rings)
lib/features/onboarding_auth/	Onboarding flow and authentication screens
lib/features/workout/	Daily workout list, exercise detail, and timer logic
lib/features/nutrition/	Smart nutrition list and meal detail screens
lib/features/profile/	Profile stats, avatar store, and settings
lib/features/dashboard/	Main dashboard and level-progression timeline

4.3 Core Data Models

Model	Key Fields
UserModel	id, name, email, level, xp, points, streakDays
WorkoutModel	id, muscleGroup, exercises[], isRestDay
ExerciseModel	id, name, sets, reps, thumbnailUrl, videoUrl
MealModel	id, title, priceEGP, calories, protein, carbs, fats, prepTime
AvatarModel	id, name, pointsCost, isUnlocked, imageUrl
LevelProgressModel	currentLevel, completionPercent, unlockDate, requiredXp

4.4 App Navigation Flow

The flow begins with the onboarding PageView, followed by level selection, then routes into authentication (Sign In / Create Account / Reset Password). After a successful login, users land on the Dashboard, which acts as the hub — from there they branch into Workout, Nutrition, Profile, and the Journey/Levels timeline, all sharing a persistent bottom-navigation shell.

4.5 State Management Approach

State is split into two tiers: local UI state (timer ticking, form validation, toggle switches) managed within each widget, and shared app state (XP, points, level, streak, unlocked avatars) managed centrally so any screen — Dashboard, Profile, or Journey — reflects updates immediately after a workout or a purchase.

5. UI/UX Design & Prototyping

5.1 Design System

All screens draw from a single AppColors theme file, keeping the palette, spacing, and typography consistent across every feature module without hardcoded values scattered through the codebase.

Token	Usage
Primary / Accent	Buttons, progress ring fill, XP badge, active states
Background / Surface	Screen backgrounds and card surfaces
Text Primary / Secondary	Headings vs. supporting captions
Success / Warning	Streak protection alerts, completion confirmations

5.2 Key Screens

Screen	Purpose
Onboarding PageView	Introduces the app value proposition and collects fitness level
Sign In / Create Account	Validated authentication entry points
Dashboard Home	Level status, circular progress ring, streak, weekly checklist
Today's Workout	Muscle-group grouped exercise cards with quick-start timers
Exercise Detail	Video placeholder, full exercise info, active timer
Smart Nutrition	Meal list with price and calorie badges
Meal Details	Macro breakdown cards and preparation steps
Profile	Stat badges and active avatar display
Avatar Store	Point-based avatar unlock marketplace
My Journey & Levels	Vertical timeline with locked/unlocked milestones
Settings	Notifications, theme, version, logout

5.3 UI/UX Guidelines

- Card-based layouts for scannability across workout, nutrition, and avatar lists.
- Consistent iconography for muscle groups, XP, and points across every screen.
- Clear locked vs. unlocked visual states on the Journey timeline and Avatar Store.
- Accessible contrast between the accent color and background surfaces.

6. Implementation

6.1 Technology Stack

Layer	Technology
Client Framework	Flutter (Dart)
State Management	Shared app-level state controller for XP / points / level data
Local Persistence	Local storage for streaks, unlocked avatars, and session data
Networking	REST-based API client for authentication and meal/workout data
Version Control	Git, hosted on GitHub

6.2 Coding Standards & Naming Conventions

- Consistent lowerCamelCase for variables/methods and UpperCamelCase for classes and widgets.
- One feature per folder; each folder groups its own screens, widgets, and models.
- All colors and text styles referenced from AppColors — no hardcoded hex values in UI code.
- Meaningful, descriptive names for widgets that mirror their role (e.g., ExerciseTimerCard).

6.3 Version Control & Git Workflow

- Always pull the latest changes from Dev before starting new work.
- Create a dedicated feature branch off Dev (e.g., feature/workout, feature/dashboard).
- No direct pushes to Dev — every feature is merged through a reviewed Pull Request.
- Commit messages describe the change and the affected module.

6.4 Security & Error Handling

- Client-side input validation on all authentication and form fields.
- Graceful empty/error states for network calls (e.g., failed meal or workout fetch).
- Safe-guarded point deductions in the Avatar Store to prevent negative balances.

7. Testing & Quality Assurance

7.1 Test Plan

Testing focuses on functional correctness of the gamification logic (XP/points/streaks), form validation, and consistent rendering of the shared theme across devices and screen sizes.

7.2 Sample Test Cases

Test Case	Steps	Expected Result
Level unlock timing	Complete Level 1 requirements, advance the app clock 10 days	Level 2 becomes unlocked
Workout completion reward	Finish all exercises in a daily routine	XP and points increase by the defined amount
Avatar purchase	Select an avatar costing less than current points	Points deducted, avatar marked as unlocked
Insufficient points	Select an avatar costing more than current points	Purchase is blocked with a clear message
Rest day display	Open Today's Workout on a scheduled rest day	Rest Day layout is shown instead of an exercise list
Form validation	Submit Create Account with an invalid email	Inline validation error is shown, submission blocked

7.3 Bug Tracking

Issues discovered during manual QA are logged with steps to reproduce, expected vs. actual behavior, and the feature module affected, then resolved through the same branch-and-PR workflow used for feature work.

8. Deployment & Execution

8.1 Build & Release

- Debug builds used throughout development for rapid iteration and testing.
- A release APK is generated for final submission and demonstration.

8.2 README Contents

- Installation steps and Flutter/Dart SDK version requirements.
 - Configuration instructions (API base URL, environment values).
 - Run instructions for local execution and the location of the built APK.
-

9. Final Presentation & Reports

- User Manual — a short walkthrough of onboarding, workouts, nutrition, and the level system.
- Technical Documentation — this document, covering architecture, requirements, and design.
- Project Presentation — summary of the concept, challenges, and outcomes.
- Optional short demo video showcasing the core user flow end-to-end.

VOZ demonstrates a complete, feature-based Flutter application: gamified fitness tracking backed by a clean architecture, a consistent design system, and a workflow disciplined enough to support parallel development across multiple modules.